

연결 리스트(Linked List) 정리

대학교 강의 스크립트를 기반으로 작성된 연결 리스트 개념 정리입니다. 각 리스트 구조의 특징과 예제를 포함합니다.

연결 리스트란?

- 데이터를 메모리 상에 **비연속적으로 저장**하는 자료구조.
- 각 데이터 단위인 **노드(Node)**는 다음 노드를 가리키는 **포인터**를 포함함.
- 노드는 필요할 때마다 **동적 메모리 할당**을 통해 생성됨.
- 연결 리스트는 항상 **헤드 포인터(Head Pointer)**로 시작함.
→ 헤드 포인터는 첫 번째 노드를 가리키는 역할.

노드(Node)의 구조

```
typedef struct Node {  
    int data;           // 데이터 필드  
    struct Node* next;  // 다음 노드를 가리키는 포인터  
} Node;
```

- 각 노드는 다음과 같은 두 부분으로 구성:
 - **데이터(Data)**: 실제 저장 값
 - **링크(Link)**: 다음 노드를 가리키는 포인터

연결 리스트의 종류

1. 단순 연결 리스트 (Singly Linked List)

- 노드가 **한 방향(→)**으로만 연결됨.
- 마지막 노드의 **next**는 **NULL**.

구조 예시

```
[HEAD] → [10 | ] → [20 | ] → [30 | NULL]
```

예제 코드

```
Node* head = (Node*)malloc(sizeof(Node));  
head->data = 10;
```

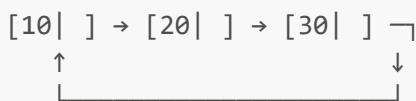
```
head->next = (Node*)malloc(sizeof(Node));
head->next->data = 20;

head->next->next = NULL;
```

2. 원형 연결 리스트 (Circular Linked List)

- 마지막 노드가 다시 첫 번째 노드(head) 를 가리킴.
- 리스트의 끝이 없음.

구조 예시



예제 코드

```
Node* head = (Node*)malloc(sizeof(Node));
Node* second = (Node*)malloc(sizeof(Node));

head->data = 10;
second->data = 20;

head->next = second;
second->next = head; // 마지막 노드가 head를 가리킴
```

3. 이중 연결 리스트 (Doubly Linked List)

- 각 노드가 앞/뒤 노드 모두와 연결됨.
- 포인터 2개 필요: prev, next

구조체 정의

```
typedef struct DNode {
    int data;
    struct DNode* prev;
    struct DNode* next;
} DNode;
```

구조 예시

```
NULL ← [10] ↔ [20] ↔ [30] → NULL
```

예제 코드

```
DNode* node1 = (DNode*)malloc(sizeof(DNode));
DNode* node2 = (DNode*)malloc(sizeof(DNode));

node1->data = 10;
node1->prev = NULL;
node1->next = node2;

node2->data = 20;
node2->prev = node1;
node2->next = NULL;
```

⚙️ 노드 생성 및 연결 요약

노드 생성 (단일)

```
Node* newNode = (Node*)malloc(sizeof(Node));
if (newNode == NULL) {
    printf("메모리 할당 실패\n");
    exit(1);
}
newNode->data = 10;
newNode->next = NULL;
```

노드 연결

```
head->next = newNode;
```

🧠 개념 요약

리스트 종류	연결 방향	마지막 노드	포인터 수
단순 연결 리스트	한 방향(→)	NULL	1 (next)
원형 연결 리스트	한 방향(→)	head (첫 노드)	1 (next)
이중 연결 리스트	양 방향(↔)	앞뒤 모두 연결	2 (prev, next)

⚠️ 주의사항

- 동적 메모리 할당 후 반드시 **NULL** 체크 필요
 - 연결 해제 시 **free()** 함수로 메모리 해제
 - 연결 시 실수로 무한 루프 발생 가능 → 종료 조건 필수 확인
-

☑ 정리

- 연결 리스트는 배열보다 유연한 삽입/삭제 연산이 가능함.
 - 동적 메모리를 사용하여 필요할 때만 메모리 사용.
 - 구조체, 포인터, 메모리 할당을 적절히 활용해야 함.
 - 다음 시간에는 삽입, 삭제, 탐색 등의 연결 리스트 연산 구현을 배움.
-

📄 작성자: ChatGPT, 대학 강의 기반 정리

🕒 최종 업데이트: 2025-03-29